SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**192524358**
Name	**Abdul Razak**

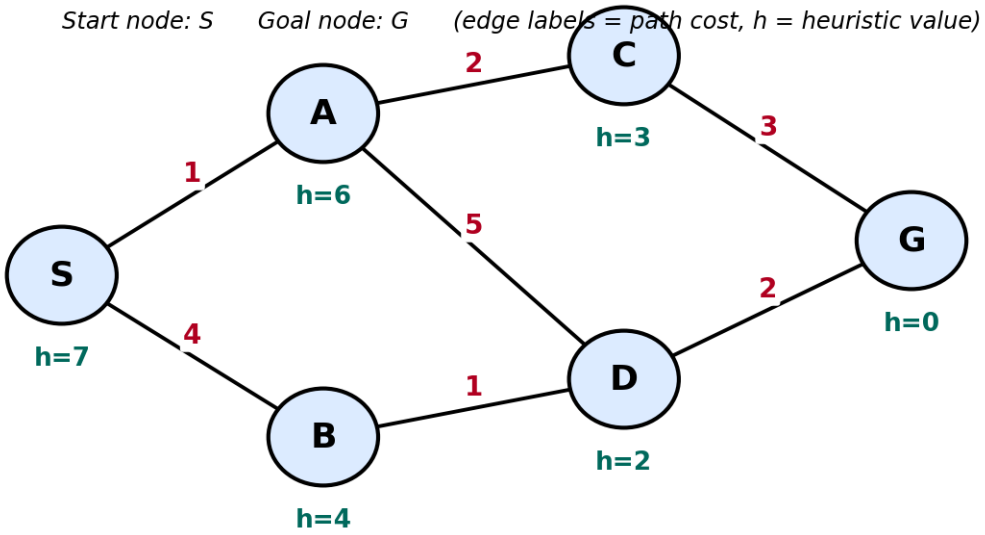
COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

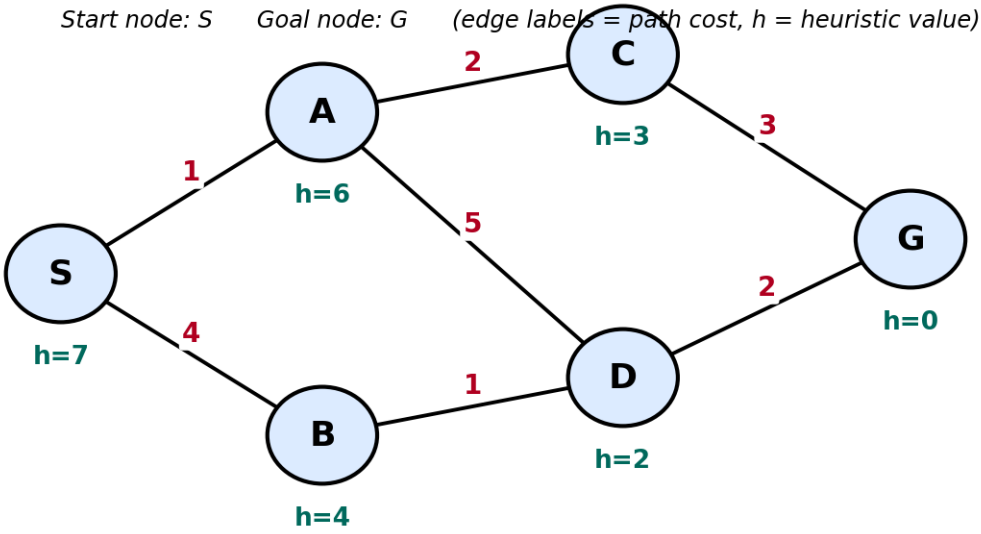
QUESTIONS: 5 **Total Marks:50**

Q1. Consider a suitable state-space graph (with heuristic values h(n) assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.

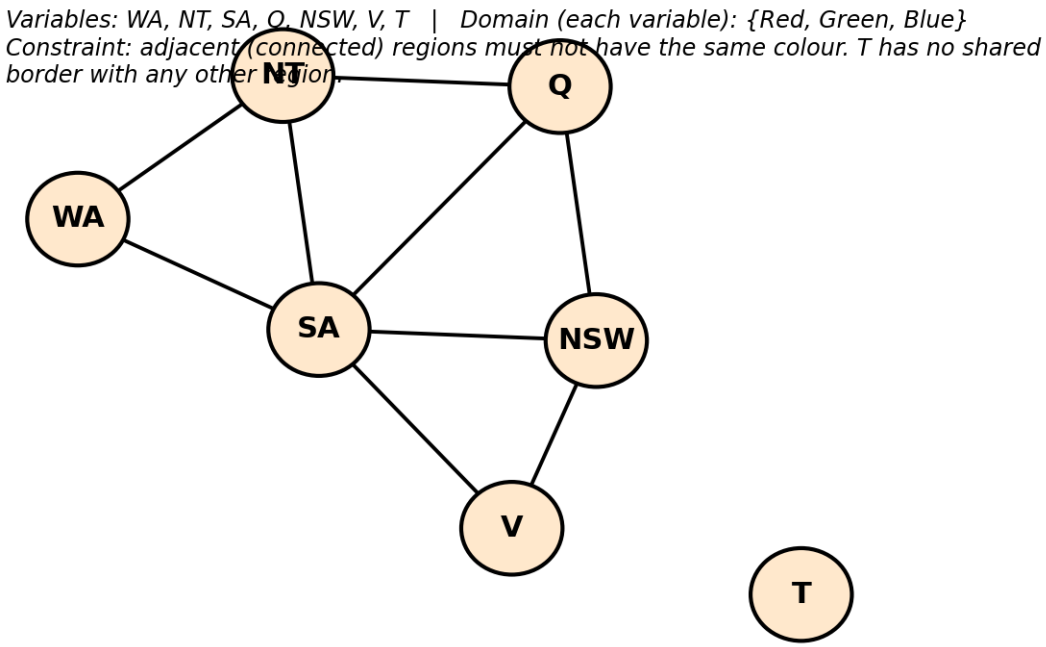


Q2. For a suitable weighted state-space graph (with edge costs and heuristic values h(n)) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the g(n), h(n) and f(n) values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.

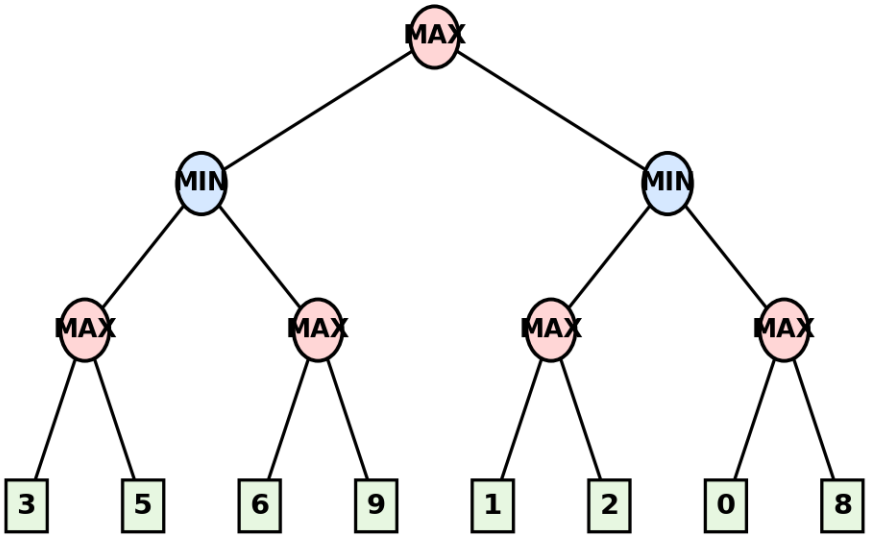


Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.



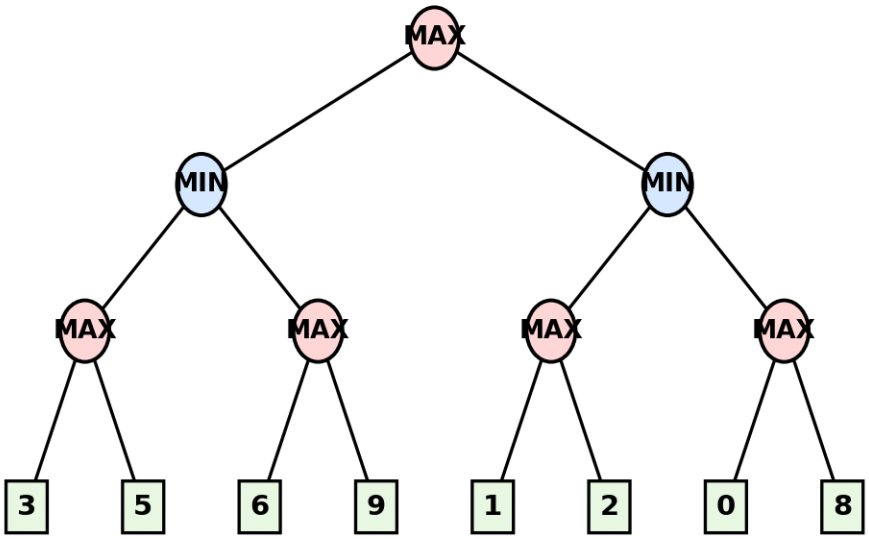
Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I _____Shaik. Abdul Razak (192524358_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik. Abdul Razak

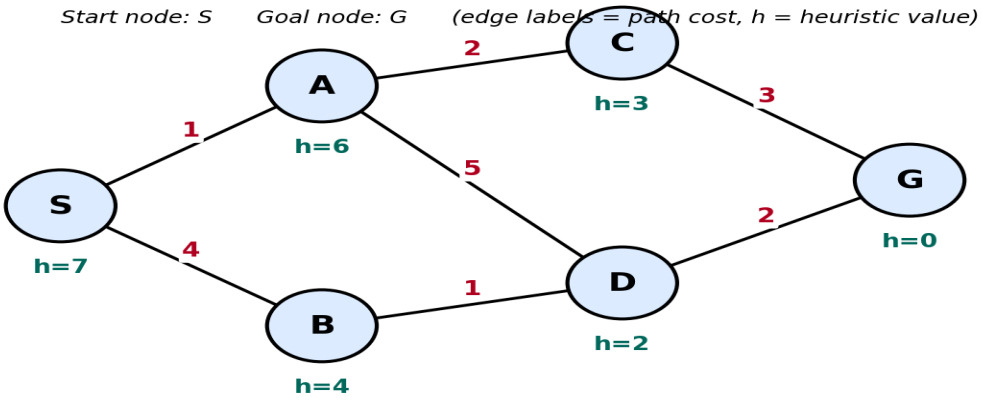
Dry Run Evaluation

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	**10**
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	**10**
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α–β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the	Multiple calculation errors are present that affect the correctness of	Calculations are mostly incorrect or missing.	**10**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
		final outcome.	intermediate steps.		
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	**10**
Clarity of Presentation & Justification (trace tables/trees/ diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	**10**
				Total	50

1.Greedy Best-First Search (GBFS) Dry Run:



Algorithm Used: Greedy Best-First Search
Evaluation Function:

f(n)=h(n)f(n) = h(n)f(n)=h(n)

where **h(n)** is the heuristic value of the node. The node with the **lowest heuristic value** is selected for expansion.

Given Graph:

Node	Heuristic h(n)
S	7
A	6
B	4
C	3
D	2
G	0

Step-by-Step Dry Run

Step 1

- **Open List:** {S(7)}
- **Expand:** S

Children of S:

- A (h = 6)
- B (h = 4)

Open List after expansion: {B(4), A(6)}

Step 2

- **Select node with minimum h: B(4)**
- **Expand:** B

Child of B:

- D (h = 2)

Open List: {D(2), A(6)}

Step 3

- **Select node with minimum h: D(2)**
- **Expand:** D

Children:

- G (h = 0)
- A (already in Open List)

Open List: {G(0), A(6)}

Step 4

- **Select node with minimum h: G(0)**

Goal node reached.

Search stops.

Search Table:

Step	Open List (ordered by h)	Node Expanded
1	S(7)	S
2	B(4), A(6)	B
3	D(2), A(6)	D
4	G(0), A(6)	G (Goal)

Final Path Obtained:

S→B→D→G\boxed{S \right arrow B \right arrow D \right arrow G}S→B→D→G

Path Cost:

Edge costs are:

- S → B = 4
- B → D = 1
- D → G = 2

Total Cost

4+1+2=74 + 1 + 2 = \boxed {7}4+1+2=7

Result:

- **Algorithm:** Greedy Best-First Search
- **Expansion Order: S → B → D → G**
- **Final Path: S → B → D → G**
- **Total Path Cost: 7**

2. A Search Dry Run*:

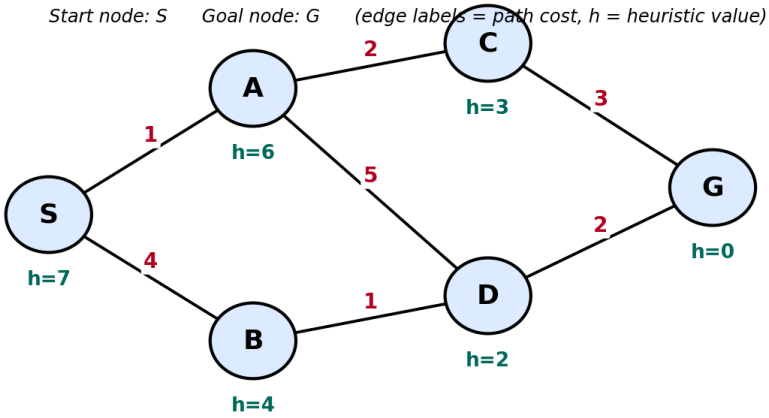
Evaluation Function

$f(n)=g(n)+h(n)f(n) = g(n) + h(n)f(n)=g(n)+h(n)$

Where:

- **g(n)** = Cost from Start node (S) to current node.
- **h(n)** = Heuristic value.
- **f(n)** = Estimated total cost.

Start node: S Goal node: G (edge labels = path cost, h = heuristic value)



Edge	Cost
S → A	1
S → B	4
A → C	2
A → D	5
B → D	1
C → G	3
D → G	2

Heuristic Values

Node	h(n)
S	7
A	6
B	4
C	3
D	2
G	0

Step 1:

Open List

Node	g(n)	h(n)	f(n)=g+h
S	0	7	**7**

Expand **S**.

Generate successors:

- A:
  - g = 0 + 1 = 1
  - h = 6
  - f = 7
- B:
  - g = 0 + 4 = 4
  - h = 4
  - f = 8

Open List:

Node	g	h	f
A	1	6	**7**
B	4	4	8

Step 2

Expand node with minimum f.

Expand A

Generate successors:

C

- g = 1 + 2 = 3
- h = 3
- f = 6

D

- g = 1 + 5 = 6
- h = 2
- f = 8

Open List:

Node	g	h	f
C	3	3	**6**
B	4	4	8
D	6	2	8

Step 3

Expand **C**

Generate successor:

G

- g = 3 + 3 = 6
- h = 0
- f = 6

Open List:

Node	**g**	**h**	**f**
G	6	0	**6**
B	4	4	8
D	6	2	8

Step 4

Expand **G**

Goal reached.

Algorithm stops.

Expansion Order

S→A→C→G

Final Optimal Path

S→A→C→G

Total Cost:

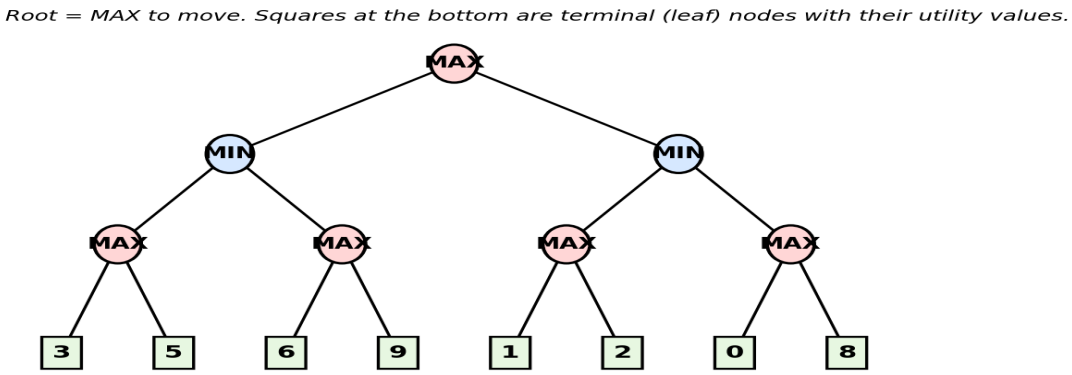
1+2+3=6

3.Minimax Algorithm Dry Run

The given game tree has **3 levels**:

- **Level 0:** MAX (Root)
- **Level 1:** MIN
- **Level 2:** MAX
- **Level 3:** Terminal (Leaf) nodes with utility values

The objective of the **MAX player** is to maximize the utility value, while the **MIN player** tries to minimize it.



Step 1: Leaf (Terminal) Node Values

The terminal node utilities are:

Left Subtree	Right Subtree
3, 5	1, 2
6, 9	0, 8

Step 2: Evaluate MAX Nodes (Bottom-Up)

Each MAX node selects the **maximum** of its children.

MAX Node 1

Children:

- 3

- 5

MAX(3,5)=5

Value = **5**

MAX Node 2

Children:

- 6
- 9

MAX(6,9)=9

Value = **9**

MAX Node 3

Children:

- 1
- 2

MAX(1,2)=2

Value = **2**

MAX Node 4

Children:

- 0
- 8

MAX(0,8)=

Value = **8**

Step 3: Evaluate MIN Nodes

Each MIN node selects the **minimum** of its MAX children.

Left MIN Node

Children:

- 5
- 9

MIN(5,9)=5

Value = **5**

Right MIN Node

Children:

- 2
- 8

MIN(2,8)=2

Value = **2**

Step 4: Evaluate Root MAX Node

Root receives:

- Left = 5
- Right = 2

MAX chooses

MAX(5,2)=5

Therefore,

Root Value = 5

Dry Run Table

Step	Node Type	Children	Computation	Value
1	MAX	3, 5	max(3,5)	**5**
2	MAX	6, 9	max(6,9)	**9**
3	MAX	1, 2	max(1,2)	**2**
4	MAX	0, 8	max(0,8)	**8**
5	MIN	5, 9	min(5,9)	**5**
6	MIN	2, 8	min(2,8)	**2**
7	MAX (Root)	5, 2	max(5,2)	**5**

Expansion Order

The Minimax evaluation is performed **bottom-up**:

1. Evaluate leaf nodes.
2. Compute MAX node values: **5, 9, 2, 8**.
3. Compute MIN node values: **5, 2**.
4. Compute the Root MAX value: **5**.

Best Move for MAX

The root MAX player compares:

- Left subtree = **5**
- Right subtree = **2**
- Since

5>25 > 25>2

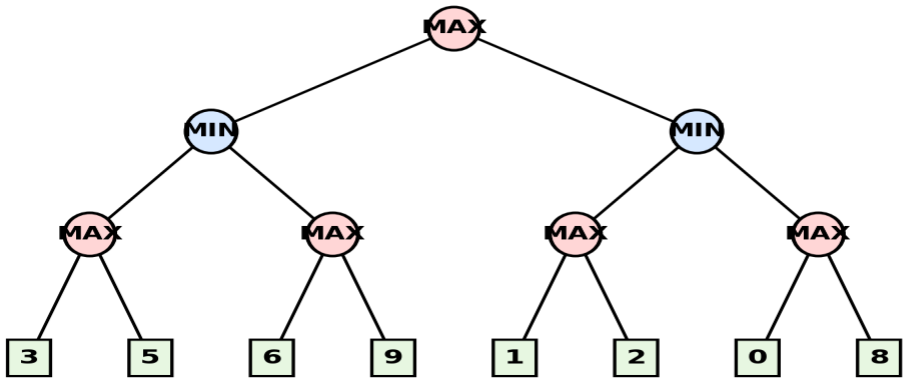
the MAX player chooses the **left branch**.

5.Minimax Algorithm with Alpha-Beta Pruning (Dry Run)

Evaluation Function

- **MAX node:** Chooses the maximum value.
- **MIN node:** Chooses the minimum value.
- **α (Alpha):** Best (highest) value found so far for MAX.
- **β (Beta):** Best (lowest) value found so far for MIN.
- **Pruning Rule:** If **α ≥ β**, the remaining branches are pruned because they cannot affect the final decision.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Given Game Tree :

Leaf Utility Values:

- Left Subtree: **3, 5, 6, 9**
- Right Subtree: **1, 2, 0, 8**

Step-by-Step Dry Run:

Step 1: Root (MAX)

- $\alpha = -\infty$
- $\beta = +\infty$

Explore the **left MIN subtree** first.

Step 2: Left MIN Node

- $\alpha = -\infty$

- $\beta = +\infty$

Explore the left MAX node.

Step 3: Left MAX Node

Leaf values: **3, 5**

- $\max(3,5) = \mathbf{5}$

Return **5**.

At Left MIN:

- $\beta = \min(+\infty,5) = \mathbf{5}$

Current values:

- $\alpha = -\infty$
- $\beta = 5$

Step 4: Second MAX Node (Left Subtree)

Receives:

- $\alpha = -\infty$
- $\beta = 5$

Evaluate first leaf:

Value = **6**

Update:

- $\alpha = \max(-\infty,6) = \mathbf{6}$

Now,

$\alpha=6, \beta=5$

Prune the remaining child (9).

Return value = **6**

Left MIN chooses

Min (5,6)=5 min(5,6)=5 min(5,6)=5

Return **5** to Root.

Step 5: Root (MAX)

Current value:

- $\alpha = \max(-\infty, 5) = \mathbf{5}$
- $\beta = +\infty$

Now explore the **right MIN subtree**.

Step 6: Right MIN Node

Receives

- $\alpha = 5$
- $\beta = +\infty$

Explore first MAX node.

Step 7: Third MAX Node

Leaf values:

- 1
- 2

Maximum = **2**

Return **2**

At Right MIN:

$\beta = \min(+\infty, 2) = \mathbf{2}$

Now,

$\alpha=5, \beta=2$

Prune the remaining MAX subtree (0, 8).

Return **2** to Root.

Step 8: Root (MAX)

Compare:

- Left = 5
- Right = 2

Choose

MAX(5,2)=5

Search ends.

Alpha-Beta Values Table

Step	**Node**	**α**	**β**	**Action**
1	Root (MAX)	−∞	+∞	Start
2	Left MIN	−∞	+∞	Explore
3	Left MAX	−∞	+∞	Returns 5
4	Left MIN	−∞	5	β updated to 5
5	Second MAX	6	5	**Prune leaf 9**
6	Root	5	+∞	α updated to 5
7	Right MIN	5	+∞	Explore
8	Third MAX	5	+∞	Returns 2
9	Right MIN	5	2	**Prune subtree (0,8)**
10	Root	5	+∞	Final Value = 5

Pruned Branches

Pruning 1

At the second MAX node:

- First child = **6**
- $\alpha = 6$
- $\beta = 5$

Since

the second leaf (**9**) is **pruned**.

Pruning 2

At the right MIN node:

- Current $\beta = 2$
- Root $\alpha = 5$

Since

the entire right MAX subtree containing leaves **0** and **8** is **pruned**.

Final Answer

- Root (MAX) Value: 5
- Best Move for MAX: Choose the left subtree
- Pruned Branches:
  - Leaf 9 (at the second MAX node)
  - Entire rightmost subtree containing 0 and 8
- Reason for Pruning: In both cases, $\alpha \geq \beta$, so those branches could not influence the final Minimax decision.
- Optimal Utility Value: 5

3.Constraint Satisfaction Problem (CSP) – Map Colouring Using Backtracking Search

Problem Formulation

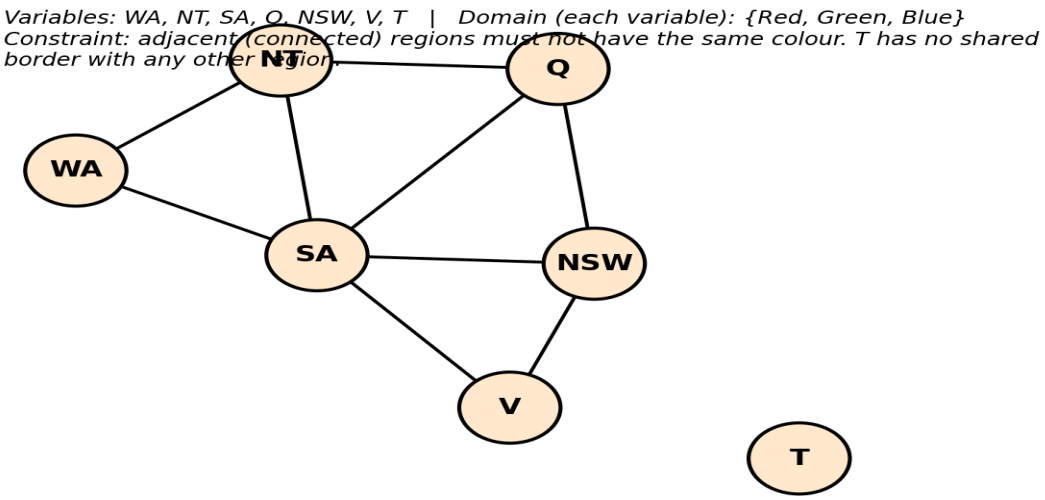
Variables

The variables represent the Australian states/territories:

{WA, NT, SA, Q, NSW, V, T}

Where:

- WA = Western Australia
- NT = Northern Territory
- SA = South Australia
- Q = Queensland
- NSW = New South Wales
- V = Victoria
- T = Tasmania



Domain

Each variable can be assigned one of the following colours:

{Red, Green, Blue}

Constraints

Adjacent (connected) regions **must not have the same colour**.

The constraints are:

- WA ≠ NT
- WA ≠ SA
- NT ≠ SA
- NT ≠ Q
- SA ≠ Q
- SA ≠ NSW
- SA ≠ V
- Q ≠ NSW
- NSW ≠ V

Note: Tasmania (T) has no neighbouring states, so it has no constraints.

Backtracking Search Dry Run

Variable Ordering

Assign variables in the following order:

WA→NT→SA→Q→NSW→V→TWA
TWA→NT→SA→Q→NSW→V→T

Step 1

Assign WA = Red

Constraint Check:

- No previously assigned neighbouring state.

Result: Valid

Current Assignment:

WA = Red

Step 2

Assign NT = Red

Constraint Check:

- WA ≠ NT
- Red = Red (Violation)

Backtrack and try the next colour.

Assign NT = Green

Constraint Check:

- WA ≠ NT
- Red ≠ Green

Result: Valid

Current Assignment:

WA = Red

NT = Green

Step 3

Assign SA = Red

Constraint Check:

- WA ≠ SA
- Red = Red (Violation)

Try the next colour.

Assign SA = Green

Constraint Check:

- NT ≠ SA
- Green = Green (Violation)

Try the next colour.

Assign SA = Blue

Constraint Checks:

- WA ≠ SA → Red ≠ Blue
- NT ≠ SA → Green ≠ Blue

Result: Valid

Current Assignment:

WA = Red

NT = Green

SA = Blue

Step 4

Assign Q = Red

Constraint Checks:

- NT ≠ Q → Green ≠ Red
- SA ≠ Q → Blue ≠ Red

Result: Valid

Current Assignment:

WA = Red

NT = Green

SA = Blue

Q = Red

Step 5

Assign NSW = Red

Constraint Check:

- Q ≠ NSW
- Red = Red (Violation)

Backtrack and try another colour.

Assign NSW = Green

Constraint Checks:

- Q ≠ NSW → Red ≠ Green
- SA ≠ NSW → Blue ≠ Green

Result: Valid

Current Assignment:

WA = Red

NT = Green

SA = Blue

Q = Red

NSW = Green

Step 6

Assign V = Red

Constraint Checks:

- NSW ≠ V → Green ≠ Red
- SA ≠ V → Blue ≠ Red

Result: Valid

Current Assignment:

WA = Red

NT = Green

SA = Blue

Q = Red

NSW = Green

V = Red

Step 7

Assign T = Red

Tasmania has no neighbouring states.

Result: Valid

Complete Solution

Variable	Assigned Colour
WA	Red
NT	Green
SA	Blue
Q	Red
NSW	Green
V	Red
T	Red

Constraint Checking Summary

Step	Variable	Colour Tried	Constraint Check	Result
1	WA	Red	No constraint	valid
2	NT	Red	WA = NT	Invalid
3	NT	Green	WA ≠ NT	valid
4	SA	Red	WA = SA	Invalid
5	SA	Green	NT = SA	Invalid
6	SA	Blue	WA ≠ SA, NT ≠ SA	valid
7	Q	Red	NT ≠ Q, SA ≠ Q	valid
8	NSW	Red	Q = NSW	Invalid
9	NSW	Green	Q ≠ NSW, SA ≠ NSW	valid
10	V	Red	NSW ≠ V, SA ≠ V	valid
11	T	Red	No constraint	valid

Backtracking Steps

1. **NT = Red** → Conflict with **WA** → Backtrack → Assign **Green**.
2. **SA = Red** → Conflict with **WA** → Backtrack.
3. **SA = Green** → Conflict with **NT** → Backtrack.
4. **SA = Blue** → Valid.
5. **NSW = Red** → Conflict with **Q** → Backtrack → Assign **Green**.

Final Colour Assignment:

WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red, T=Red